C5 - Agent Tools

EECE 503P/798S: Agentic Systems

Assignment: Building and Demonstrating an MCP Toolkit with FastMCP and the OpenAI Agents SDK

Overview

In this assignment, you will explore the Model Context Protocol (MCP), study the design principles that separate a good agent tool from a poor one, and then design and implement a small MCP toolkit made of exactly three tools: one Data tool, one Action tool, and one Orchestration tool, built with FastMCP and connected to an agent through the OpenAI Agents SDK. Finally, you will record a short screencast demonstrating your agent using all three tools in real time.

Learning Objectives

By completing this assignment, you will be able to:

- Understand the fundamentals of the Model Context Protocol (MCP) and its Host, Client, and Server architecture.
- Apply the tool design principles from class (well-defined interface, isolated functionality, predictable behavior) and explain why single-purpose tools are preferred over large multi-purpose ones.
- Implement Data, Action, and Orchestration tools as distinct MCP tools using FastMCP.
- Demonstrate and evaluate agent interactions with your MCP server in real-world scenarios.

Prerequisites

- Familiarity with Python and async programming.
- Installed OpenAI Agents SDK and FastMCP (latest versions).
- Basic knowledge of HTTP/JSON APIs and REST.
- Access to a Hugging Face account (for reading the article).

Part 1: Reading and Exploration

- Read the MCP blog post on Hugging Face: https://huggingface.co/blog/Kseniase/mcp
- Explore at least one existing MCP server implementation to see these ideas applied in practice. Good starting points from class include:
  - The Fetch MCP Server (retrieves a webpage and converts it to a model-friendly format)
  - A filesystem MCP server
  - Any other open-source MCP server you find interesting
- Write a short summary (300 to 400 words) highlighting:
  - The key MCP concepts discussed in the Hugging Face article, including how the Host, Client, and Server roles relate to one another.
  - How the server you explored is designed: is each tool single-purpose and well documented, or does it bundle multiple responsibilities together? Critique it using the tool design principles from class.

Part 2: Implementation

Choose a theme for your toolkit. It can be inspired by an existing service or fully original. Some ideas:

- A mapping or location assistant (geocoding, routing, points of interest)
- A weather and travel planning assistant
- A campus or library lookup assistant
- A personal productivity assistant (notes, reminders, scheduling)

Within your chosen theme, you must design exactly three tools, one from each category covered in class:

- **Data tool:** retrieves or looks up information without changing anything (for example, geocode_address, get_weather_forecast, search_catalog).
- **Action tool:** changes something or performs an external effect (for example, save_favorite_location, create_reminder, send_notification).
- **Orchestration tool:** coordinates the other two, calling them in sequence or combining their results to accomplish a higher-level request (for example, plan_trip, which looks up a route and then saves it as a favorite).

For each tool:

- Give it a clear textual description, typed arguments, and a typed or documented output.
- Keep it single-purpose. If you find yourself describing a tool with "and", consider splitting it into two tools coordinated by your Orchestration tool instead.
- Handle at least one predictable failure case (for example, invalid input or an unavailable resource) with a clear error rather than a silent failure.

Build the Server and Client

- Implement your three tools as an MCP server using FastMCP (@mcp.tool for callable capabilities, and @mcp.resource for any pure data lookups you expose as resources).
- Connect your MCP server to an agent built with the OpenAI Agents SDK, acting as the MCP Host/Client, so the agent can route user requests to the correct tool.
- Write unit tests or simple interactive scripts to verify each tool's functionality, including the failure case above.

Part 3: Demonstration

Record a 5 to 7 minute screencast where you:

- Briefly introduce your project and the three tools you built, naming which is the Data tool, which is the Action tool, and which is the Orchestration tool.
- Show your agent in action: issue queries and demonstrate the full round trip from the agent, to the MCP client, to your MCP server, and back.
- Highlight any challenges faced (for example, keeping tools single-purpose, handling failures predictably) and how you overcame them.
- Share your code repository (for example, a GitHub link) and the video recording in your submission.

Deliverables

- **Written Summary:** of the article and the existing MCP server you explored (PDF or Markdown).
- Source Code for your agent project, including:
  - A FastMCP server module implementing your three tools.
  - The OpenAI Agents SDK client/host code that connects to your server.
  - Configuration and setup instructions (README).

    - Tests or example scripts, including your failure case.
- **Screencast Video:** demonstrating your agent (hosted on a platform of your choice, for example YouTube unlisted or a Zoom recording).
- **Reflection:** (1 to 2 paragraphs) on how you decided where to draw the line between your three tools, whether three tools felt like the right number for your use case, and potential next steps.